

Basic JSON Blog

BackEnd – NodeJs + Express

FrontEnd – HTML, CSS, JS

Requirements:

- **What:** API to create, read, and delete blog posts stored in memory.
- **Skills:** CRUD operations, handling JSON bodies.
- **Stretch:** Add timestamps and basic search functionality.

BackEnd:

Express is used, on `PORT = 8000;`

As well these two middlewares:

```
app.use(express.json());
```

(this to send/retrieve json objects)

```
app.use(express.static("public"));
```

(this so that express knows where to search for index.html and style.css that are present in folder “public”)

in this case, myBlogs (array of JSON objects) is used as a mock up db; how an object looks like:

```
{  
  id: Date.now(),  
  title: "Title 1",  
  createdAt: new Date().toISOString(),  
  updatedAt: new Date().toISOString(),  
  image: "img.png",  
  textContent: ["text1", "text2"],  
}
```

CRUD → Create/POST; Read/GET; Update/PATCH; D/Delete

We make a route `get /api/blogs` to retrieve a request query from search, if the search is empty, we can give all `myBlogs` as it is unfiltered, if the search has a certain title or text that when filtered appears on `myBlogs`, we send back the response `myBlogs` with filtered items that were found using search

```
app.get("/api/blogs", (request, response) => {  
  const { search } = request.query;
```

```
  if (!search) {  
    return response.status(200).json(myBlogs);  
  }
```

```
  const filteredBlogs = myBlogs.filter((blog) => {  
    const titleMatch = blog.title.toLowerCase().includes(search.toLowerCase());
```

```
    const textMatch = blog.textContent.some((text) =>  
      text.toLowerCase().includes(search.toLowerCase()),  
    );
```

```
    return titleMatch || textMatch;  
  });
```

```
  response.status(200).json(filteredBlogs);  
});
```

We make a route post /api/blogs to create a new JSON object based on what we received from the frontend (request.body). The request.body is deserialized as properties that will be used to create the new JSON object and which later be pushed into myBlogs

```
app.post("/api/blog", (request, response) => {
  console.log(request.body);

  const { title, image, textContent } = request.body;

  const newBlog = {
    id: Date.now(),
    title,
    image,
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
    textContent,
  };

  myBlogs.push(newBlog);

  response.status(201).json(newBlog);
});
```

We make a route delete /api/blog/:id we get a request.params.id from the front end, if that id exists, meaning if using the high order function find(id) on myBlogs and we find an id with that exists in a JSON objects, we will delete it by filtering myBlogs by excluding that blog with that certain id.

```
app.delete("/api/blog/:id", (request, response) => {
  const foundID = parseInt(request.params.id);
  const thatBlog = myBlogs.find((blog) => blog.id === foundID);
  if (!thatBlog) {
    response.status(404).json({ message: `Blog with ${foundID} not found` });
  } else {
    myBlogs = myBlogs.filter((blog) => blog.id !== foundID);
    response
      .status(200)
      .json({ message: `${thatBlog} was deleted sucessfully` });
  }
});
```

we use patch /api/blogs/ to patch a certain blog with a certain id. Again the myBlogs.find(id) is used to see if that blog with that id exists.

If the blog with that id exists, we create a new updateBlog that is a frankenstein mix of the original blog (thatBlog) and the override with the new request.body we get from the front end. When the updateBlog is done, we replace it inside the myBlogs using splice method

```
app.patch("/api/blog/:id", (request, response) => {
  const foundID = parseInt(request.params.id);
  const thatBlog = myBlogs.find((blog) => blog.id === foundID);
  if (!thatBlog) {
    response.status(404).json({ message: `Blog with ${foundID} not found` });
  } else {
    const updatedBlog = {
      ...thatBlog,
      title: request.body.title || thatBlog.title,
      image: request.body.image || thatBlog.image,
      textContent: request.body.textContent || thatBlog.textContent,
      updatedAt: new Date().toISOString(),
    };

    // deletes from index, amount, inserts new elements at that index
    const oldIndexOffset = myBlogs.indexOf(thatBlog);
    myBlogs.splice(oldIndexOffset, 1, updatedBlog);

    response.status(200).json(updatedBlog);
  }
});
```

FrontEnd:

what matters the most in this index.html : `<div id="mainBlog" class="leftcolumn">`

1) this is where we will populate the JSON object that we will get from the server side (myBlogs), the pattern:

```
<div class="card postBlog">
<h2>id</h2>
<h2>TITLE HEADING</h2>
<h5>Dec 7, 2017</h5>
<div class="fakeimg" style="height: 200px">Image</div>
<p>Some text..</p>
<p>
Sunt in culpa qui officia deserunt mollit anim id est laborum
consectetur adipiscing elit, sed do eiusmod tempor incididunt ut
labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
exercitation ullamco.
</p>
</div>
```

2) The search input + search button

inside mainBlog we also have:

```
<input id="searchInput" placeholder="Search blog..." />
<button onclick="searchBlogs()">Search</button>
```

that will be used to search for the blog;

3) the Add new Blog input _ submit button (to add/create new Blog)

```
<div id="addBlog" class="card">
<h3>Create New Blog</h3>
<input id="newTitle" placeholder="Title" />
<input id="newImage" placeholder="Image" />
<textarea id="newText" placeholder="Text"></textarea>
<button onclick="createBlog()">Create</button>
</div>
```

4) <script>

3 constants are created: mainBlog, createNewBlog and searchValue (using document.getElementById("x")

searchValue will be used in the async function searchBlogs()

to fetch the get route /api/blogs?search=\${searchValue} so that we can filter based on title or textContent (please see app.get("api/blogs"))

mainBlog will be used in the async function getAllBlogs()

to fetch the get route /api/blogs

the backend responds with an array of blog object back (the whole mainBlog),

blogServer = await response.json = mainBlog

```
mainBlog.querySelectorAll(".postBlog").forEach((el) => el.remove());
```

that is being used to always render fresh with no duplicates

```
blogServer.forEach((blog) => renderBlog(blog));
```

than this piece of code just loops for each blog element from blogServer, calls renderBlog(blog) for each one

renderBlog() function just creates html elements

the blog will have a title, date, image placeholder, delete and update buttons

the delete button is this async function

```
async function deleteBlogById(id, blogContainer) {  
  try {  
    const response = await fetch(`/api/blog/${id}`, {  
      method: "DELETE",  
    });  
  };
```

```
  if (response.ok) {  
    blogContainer.remove();  
  } else {  
    console.log("Failed to delete");  
  }  
  catch (err) {  
    console.error("Delete error", err);  
  }  
}
```

fetches the delete method for route /api/blog/\${id}

if that element was found, the blogContainer html is removed, as well the JSON object from the server side

the update button

```
async function updateBlogById(blog) {
  const newTitle = prompt("Edit title", blog.title);
  const newImage = prompt("Edit image", blog.image);
  const newText = prompt("Edit text", blog.textContent.join("\n"));
  const updatedTextArray = newText ? newText.split("\n") : blog.textContent;

  if (!newTitle && !newText && !newImage) return;

  try {
    const response = await fetch(`/api/blog/${blog.id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        title: newTitle || blog.title,
        image: newImage || blog.image,
        date: new Date().toISOString(),
        textContent: updatedTextArray,
      }),
    });

    if (response.ok) {
      getAllBlogs();
    }
  } catch (err) {
    console.error("Update error", err);
  }
}
```

also is a fetch api to PATCH /api.blog/\${id} we are giving to server a JSON stringify with the title, image, textContent, if the server responds back with ok, it the async function will call the other async getAllBlogs() function,

else an error will be given

in order to add new blogs, we have that section

```
<div id="addBlog" class="card">
  <h3>Create New Blog</h3>
  <input id="newTitle" placeholder="Title" />
  <input id="newImage" placeholder="Image" />
  <textarea id="newText" placeholder="Text"></textarea>
  <button onclick="createBlog()">Create</button>
</div>
```

```
async function createBlog() {
  const title = document.getElementById("newTitle").value;
  const image = document.getElementById("newImage").value;
  const text = document.getElementById("newText").value;
```

```
  if (!title || !text) {
    alert("Title and text are required");
    return;
  }
```

```
  try {
    const response = await fetch("/api/blog", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        title,
        image,
        textContent: [text],
      }),
    });
  }
```

```
  if (response.ok) {
    document.getElementById("newTitle").value = "";
    document.getElementById("newImage").value = "";
    document.getElementById("newText").value = "";
  }
```

```
  getAllBlogs();
} catch (err) {
  console.error("Create error", err);
}
}
```


when we write on those inputs new text and then click on Create, the async function `createBlog()` will try to fetch an api post `/api/blog`, and we send to body a JSON stringify with the title, image, `textContent`,

the server will add the request body with what they got, create the JSON object and push it into the mock db,

if everything went well, response is ok, the inputs from `<div id="addBlog" class="card">` are reseted and we call again the `getAllBlogs()`